PROJETO INTEGRADOR TRANSDISCIPLINAR (PIT) EM CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO TÉCNICO

Catálogo Simples de Livros

Aplicação web em Java com Servlets, JSP e JDBC

Aluno: WILLIAM RIOS PEREIRA

RGM: 34417877

Data: 10/11/2026

SUMÁRIO

1. INTRODUÇÃO

2. METODOLOGIA

2.1 Abordagem de desenvolvimento

2.2 Requisitos

2.3 Tecnologias utilizadas

3. MODELAGEM DO SISTEMA

3.1 Diagrama de casos de uso

3.2 Diagrama de classes

3.3 Modelo de dados (DER) e script SQL

4. DESENVOLVIMENTO E IMPLEMENTAÇÃO

4.1 Arquitetura geral

4.2 Funcionalidade-chave 1: Cadastro

4.3 Funcionalidade-chave 2: Busca por título ou autor

4.4 Documentação do código-fonte

4.5 Repositório e documentação do projeto

5. RESULTADOS

6. CONSIDERAÇÕES SOBRE SEGURANÇA

7. CONCLUSÕES

8. REFERÊNCIAS

APÊNDICE A — MANUAL DO USUÁRIO

1. INTRODUÇÃO

Este projeto consiste em uma aplicação web para catalogar e consultar livros. O usuário pode cadastrar, listar, ver detalhes, editar, excluir e buscar livros por título ou autor, e os dados ficam salvos em um banco de dados relacional.

O código-fonte completo e toda a documentação do projeto estão publicados no GitHub, em https://github.com/wiirios/pit-cc (seção 4.5).

2. METODOLOGIA

2.1 Abordagem de desenvolvimento

Foi adotada uma abordagem **iterativa**: o sistema foi construído em partes, e cada iteração terminava com a aplicação rodando e sendo testada antes de começar a próxima.

Iteração	Entrega	Testes ao final
1	Configuração do projeto Maven (war, Java 17), conexão com o banco e criação automática da tabela	Compilação e subida do servidor; criação do arquivo do banco
2	Listagem e cadastro (DAO, Service com validação, Servlets e JSPs)	Cadastro válido, campos vazios, ano inválido, tamanhos máximos, XSS e SQL Injection
2b	Refinamento de escopo: catálogo somente de livros (remoção do tipo de mídia; tabela `livro`)	Repetição dos testes da iteração 2 com o novo modelo
3	Detalhes, edição, exclusão com confirmação e busca por título ou autor	Busca (acentos, maiúsculas, `%`, SQL Injection), ids inválidos, edição e exclusão
4	Auditoria contra o enunciado, correções e documentação	Roteiro completo (`docs/roteiro-de-testes.md`)

2.2 Requisitos

Funcionais

ID	Requisito
RF01	Interface web para navegação e gerenciamento
RF02	Cadastrar livro (título, autor, ano de publicação, gênero, sinopse)
RF03	Listar todos os livros
RF04	Ver os detalhes de um livro
RF05	Editar um livro
RF06	Excluir um livro (com confirmação)
RF07	Buscar por parte do título ou do autor
RF08	Persistir os dados em banco de dados

Não funcionais

ID	Requisito	Como foi atendido
RNF01	Usabilidade: interface simples e intuitiva	Menu fixo, mensagens de sucesso/erro em português, formulário que mantém os dados após um erro
RNF02	Responsividade	CSS próprio com *media query*: em telas de até 640 px cada livro da estante vira um cartão, e a lombada deita acima da ficha nos detalhes
RNF03	Desempenho razoável	Uma consulta SQL por tela; sem bibliotecas pesadas no navegador

ID	Requisito	Como foi atendido
RNF04	Segurança	PreparedStatement, `<c:out>`, validação no servidor, páginas de erro amigáveis (seção 6)
RNF05	Facilidade de execução	Um único comando (`mvn jetty:run`); o banco é criado automaticamente

2.3 Tecnologias utilizadas

Tecnologia	Versão	Papel
Java (JDK)	17 (`maven.compiler.release`)	Linguagem
Apache Maven	3.9	Build e dependências; empacotamento `war`
Jakarta Servlet	6.0	Controladores (`@WebServlet`)
JSP + EL + JSTL	JSTL 3.0	Visão (telas)
JDBC	(Java SE)	Acesso ao banco com `PreparedStatement`
H2 Database	2.2.224, modo arquivo	SGBD relacional
Eclipse Jetty	12.0.16 (`jetty-ee10-maven-plugin`)	Servidor web, iniciado por `mvn jetty:run`
HTML5 + CSS3	—	Interface, sem frameworks

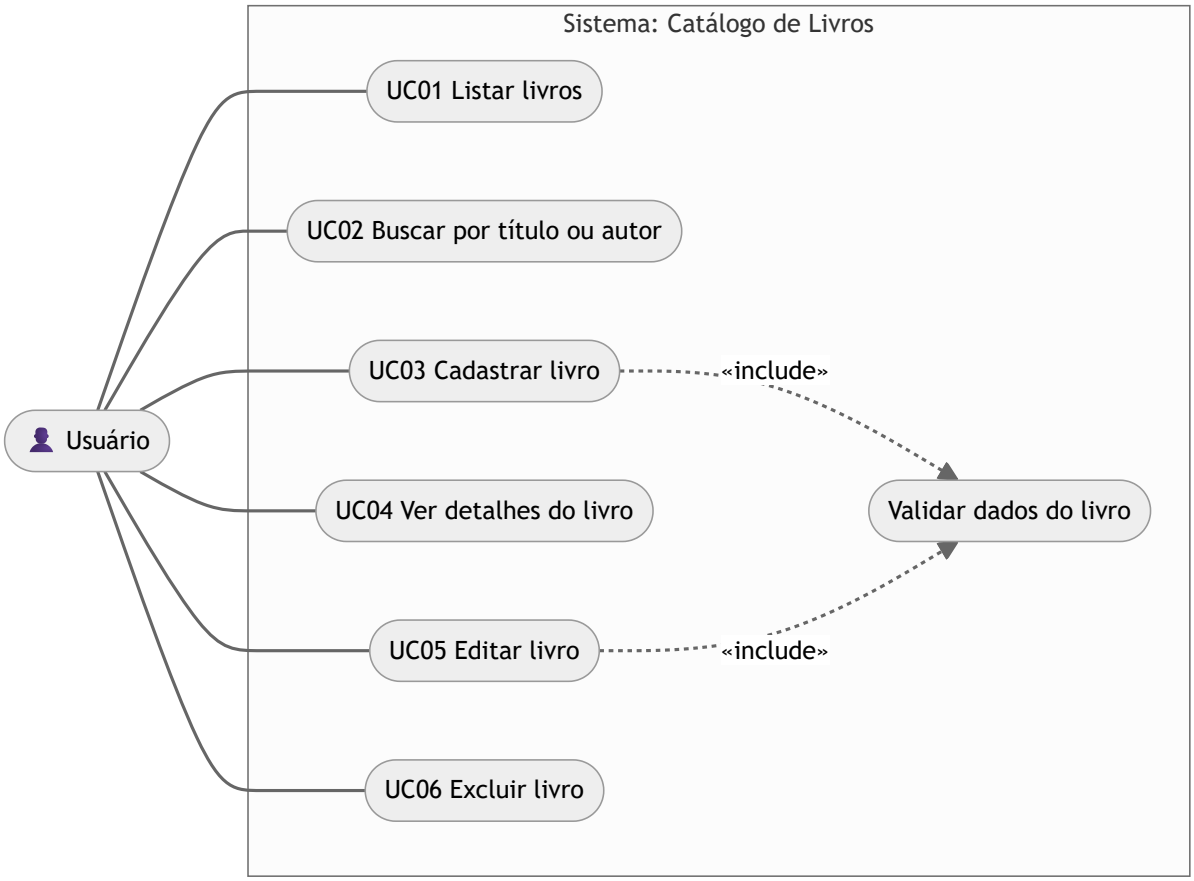
Por que H2: o enunciado pede um banco relacional, "como MySQL, PostgreSQL etc.". O H2 é relacional e usa SQL padrão via JDBC, como os outros, mas roda embutido na aplicação. Assim não é preciso instalar um servidor de banco para executar o projeto. Para migrar para MySQL, bastaria trocar a URL em `FabricaDeConexoes` e o driver no `pom.xml`.

3. MODELAGEM DO SISTEMA

Os diagramas completos, em formato editável (Mermaid), estão em `docs/modelagem.md`.

3.1 Diagrama de casos de uso

Figura 1 – Diagrama de casos de uso

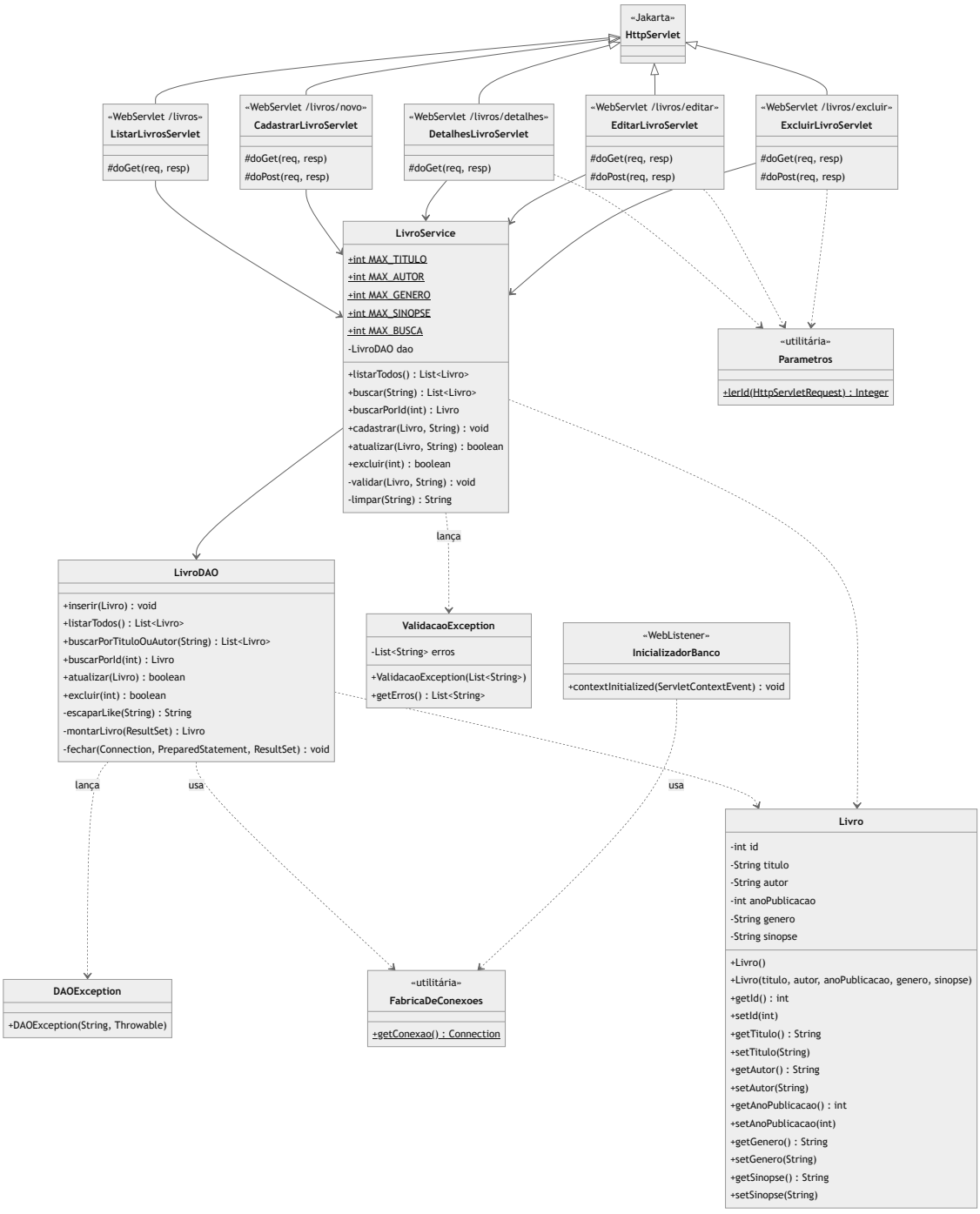


Fonte: elaborado pelo autor (2026).

Um único ator (Usuário) com seis casos de uso: Listar, Buscar, Cadastrar, Ver detalhes, Editar e Excluir. Cadastrar e Editar incluem "Validar dados do livro".

3.2 Diagrama de classes

Figura 2 – Diagrama de classes



Fonte: elaborado pelo autor (2026).

- **Modelo:** `Livro`, com atributos privados, getters/setters e dois construtores (encapsulamento).

- **Controle:** `ListarLivrosServlet`, `CadastrarLivroServlet`, `DetalhesLivroServlet`, `EditarLivroServlet` e `ExcluirLivroServlet`, que herdam de `HttpServlet` e sobrescrevem `doGet`/`doPost`.
- **Regras de negócio:** `LivroService` (validação e decisões).
- **Acesso a dados:** `LivroDAO` (único lugar com SQL de negócio).
- **Apoio:** `FabricaDeConexoes` (conexão), `InicializadorBanco` (cria a tabela ao subir), `Parametros` (leitura segura do id), `DAOException` e `ValidacaoException`.

3.3 Modelo de dados (DER) e script SQL

Figura 3 – Diagrama Entidade-Relacionamento (DER) da tabela livro

LIVRO			
INT	id	PK	AUTO_INCREMENT
VARCHAR_255	titulo		NOT NULL
VARCHAR_255	autor		NOT NULL
INT	ano_publicacao		NOT NULL
VARCHAR_100	genero		opcional
VARCHAR_2000	sinopse		opcional

Fonte: elaborado pelo autor (2026).

Uma tabela, `livro`, sem relacionamentos:

```
CREATE TABLE IF NOT EXISTS livro (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    titulo         VARCHAR(255)  NOT NULL,
    autor          VARCHAR(255)  NOT NULL,
    ano_publicacao INT           NOT NULL,
    genero         VARCHAR(100),
    sinopse        VARCHAR(2000)
);
```

Os tamanhos das colunas são os mesmos limites validados no `LivroService`. O banco funciona como uma segunda barreira caso a validação falhe (folha 26 do enunciado: "embora você possa codar isso no SGBD como restrição").

4. DESENVOLVIMENTO E IMPLEMENTAÇÃO

4.1 Arquitetura geral

```
Navegador ──HTTP──► Servlet ──► LivroService ──► LivroDAO ──JDBC──► H2 (tabela livro)
    ▲                  │
    └──── HTML ◄── JSP (WEB-INF/jsp) ◄── forward
```

Camada	Pacote / pasta	Responsabilidade
Visão	`src/main/webapp/WEB-INF/jsp`	Exibir dados com EL + JSTL; nenhuma regra nem SQL
Controle	`br.edu.catalogo.servlet`	Ler a requisição, chamar o Service, escolher a tela ou redirecionar
Negócio	`br.edu.catalogo.service`	Validar e aplicar as regras
Persistência	`br.edu.catalogo.dao`	Executar o SQL com `PreparedStatement`
Modelo	`br.edu.catalogo.model`	Representar o livro

As JSPs ficam dentro de `WEB-INF`, então o navegador não consegue abri-las diretamente: toda tela passa por um Servlet. Essa separação responde à Situação-Problema 3: nenhum Servlet tem SQL, e o acesso ao banco fica em um único DAO. Esse DAO reaproveita `FabricaDeConexoes`, para abrir a conexão, e os métodos `montarLivro()` e `fechar()`, para converter cada linha em objeto e liberar os recursos.

URL	Método	Servlet	Ação
`/`	GET	(`index.jsp`)	Redireciona para `/livros`
`/livros`	GET	`ListarLivrosServlet`	Lista ou busca (`?busca=`)
`/livros/novo`	GET / POST	`CadastrarLivroServlet`	Formulário / grava
`/livros/detalhes?id=`	GET	`DetalhesLivroServlet`	Mostra um livro
`/livros/editar`	GET / POST	`EditarLivroServlet`	Formulário preenchido / grava
`/livros/excluir`	GET / POST	`ExcluirLivroServlet`	Confirmação / exclui

4.2 Funcionalidade-chave 1: Cadastro

Fluxo: o usuário preenche `formulario.jsp` → POST para `/livros/novo` → `CadastrarLivroServlet` monta um objeto `Livro` → `LivroService.cadastrar()` valida → `LivroDAO.inserir()` grava → redirecionamento para a lista com mensagem de sucesso.

JSP — campo do formulário (`formulario.jsp`, linhas 56–57). O valor é escapado com `<c:out>`, para que o texto digitado volte ao campo, depois de um erro, sem risco de XSS:

```
<input type="text" id="titulo" name="titulo" maxlength="255" required
       value="<c:out value='${livro.titulo}'/>">
```

Servlet (`CadastrarLivroServlet.java`, linhas 53–76):

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    // Necessário para ler acentos corretamente (ex.: "Ficção")
    req.setCharacterEncoding("UTF-8");

    Livro livro = new Livro();
    livro.setTitulo(req.getParameter("titulo"));
    livro.setAutor(req.getParameter("autor"));
    livro.setGenero(req.getParameter("genero"));
    livro.setSinopse(req.getParameter("sinopse"));
    String anoTexto = req.getParameter("anoPublicacao");

    try {
        service.cadastrar(livro, anoTexto);
        // Padrão Post/Redirect/Get: evita cadastro duplicado ao atualizar a página
        resp.sendRedirect(req.getContextPath() + "/livros?msg=criado");
    } catch (ValidacaoException e) {
        req.setAttribute("erros", e.getErros());
        mostrarFormulario(req, resp, livro, anoTexto);
    } catch (DAOException e) {
        req.setAttribute("erro", e.getMessage());
        mostrarFormulario(req, resp, livro, anoTexto);
    }
}
```

O Servlet não valida nem acessa o banco: ele só traduz a requisição em um objeto, delega ao Service e decide a resposta. Cada tipo de problema tem seu próprio `catch`: dados inválidos (`ValidacaoException`) ou falha no banco (`DAOException`). Nos dois casos, o usuário volta ao formulário com uma mensagem clara.

Service — validação do ano (`LivroService.java`, linhas 139–153). Responde à Situação-Problema 2 ("o campo ano pode receber strings"):

```
int anoMaximo = Year.now().getValue() + 5;
if (anoTexto.isEmpty()) {
    erros.add("O ano de publicação é obrigatório.");
} else {
    try {
        int ano = Integer.parseInt(anoTexto);
        if (ano < 1 || ano > anoMaximo) {
            erros.add("O ano de publicação deve estar entre 1 e " + anoMaximo + ".");
        } else {
            livro.setAnoPublicacao(ano);
        }
    } catch (NumberFormatException e) {
        erros.add("O ano de publicação deve conter apenas números.");
    }
}
```

Todos os erros são acumulados em uma lista e lançados juntos em uma `ValidacaoException`, para o usuário corrigir tudo de uma vez.

DAO (`LivroDAO.java`, linhas 34–63):

```
public void inserir(Livro livro) throws DAOException {
    String sql = "INSERT INTO livro (titulo, autor, ano_publicacao, genero, sinopse) "
            + "VALUES (?, ?, ?, ?, ?)";

    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet chaves = null;
    try {
        conn = FabricaDeConexoes.getConexao();
        // RETURN_GENERATED_KEYS pede ao banco o id criado pelo AUTO_INCREMENT
        stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
        stmt.setString(1, livro.getTitulo());
        stmt.setString(2, livro.getAutor());
        stmt.setInt(3, livro.getAnoPublicacao());
        stmt.setString(4, livro.getGenero());
        stmt.setString(5, livro.getSinopse());
        stmt.executeUpdate();
```

```
        chaves = stmt.getGeneratedKeys();
        if (chaves.next()) {
            livro.setId(chaves.getInt(1));
        }
    } catch (SQLException e) {
        // O erro técnico vai para o log; o usuário recebe uma mensagem simples
        LOG.log(Level.SEVERE, "Erro ao inserir livro", e);
        throw new DAOException("Não foi possível salvar o livro. Tente novamente.", e);
    } finally {
        fechar(conn, stmt, chaves);
    }
}
```

Os `?` são preenchidos por `setString`/`setInt`, então os dados nunca viram parte do comando SQL. O `finally` garante que a conexão seja fechada com ou sem erro. A `SQLException` técnica vai para o log do servidor, e o usuário recebe só a mensagem amigável.

4.3 Funcionalidade-chave 2: Busca por título ou autor

JSP — formulário de busca (`lista.jsp`, linhas 11–20). Usa GET, então o termo fica na URL (ex.: `/livros?busca=machado`):

```
<form class="busca" method="get" action="<c:url value='/livros'/>" role="search">
    <label for="busca" class="oculto">Buscar por título ou autor</label>
    <input type="search" id="busca" name="busca" maxlength="100"
           placeholder="Buscar por título ou autor..."
           value="<c:out value='${busca}'/>">
    <button type="submit" class="botao">Buscar</button>
    <c:if test="${not empty busca}">
        <a class="botao botao-secundario" href="<c:url value='/livros'/>">Limpar</a>
    </c:if>
</form>
```

Service (`LivroService.java`, linhas 48–58). Uma busca vazia devolve todos os livros, e um termo muito longo é recusado:

```
public List<Livro> buscar(String termo) throws ValidacaoException, DAOException {
    termo = limpar(termo);
    if (termo.isEmpty()) {
        return dao.listarTodos();
    }
    if (termo.length() > MAX_BUSCA) {
        throw new ValidacaoException(List.of(
                "O termo de busca deve ter no máximo " + MAX_BUSCA + " caracteres."));
    }
    return dao.buscarPorTituloOuAutor(termo);
}
```

DAO (`LivroDAO.java`, linhas 106–135):

```
public List<Livro> buscarPorTituloOuAutor(String termo) throws DAOException {
    String sql = "SELECT * FROM livro "
            + "WHERE LOWER(titulo) LIKE ? ESCAPE '\\' "
            + "OR LOWER(autor) LIKE ? ESCAPE '\\' "
            + "ORDER BY titulo";

    // Ex.: termo "machado" vira o padrão "%machado%" (contém "machado")
    String padrao = "%" + escaparLike(termo.toLowerCase()) + "%";
    ...
        stmt = conn.prepareStatement(sql);
        stmt.setString(1, padrao);
        stmt.setString(2, padrao);
        rs = stmt.executeQuery();
        while (rs.next()) {
            livros.add(montarLivro(rs));
        }
    ...
}
```

Explicação da lógica:

- `LIKE '%termo%'` encontra o termo em qualquer posição do texto. `LOWER()` nos dois lados faz a busca ignorar maiúsculas e minúsculas.

- A Situação-Problema 1 descreve um aluno que **concatenava** o termo na consulta. Aqui as colunas `titulo` e `autor` são comparadas separadamente, e o termo entra **somente** como parâmetro (`?`), nunca colado na string SQL. Por isso, uma busca por `' OR '1'='1` não retorna todos os livros.
- `escaparLike()` coloca uma barra antes de `%` e `_`. Assim, se o usuário digitar `%`, ele procura o caractere `%`, e não "qualquer coisa".

4.4 Documentação do código-fonte

O código-fonte é documentado em português, em dois níveis:

- **Javadoc** em todos os métodos públicos do `LivroDAO` e do `LivroService` e em todos os `doGet`/`doPost` dos Servlets, com `@param`, `@return` e `@throws`. Cada classe também tem um comentário explicando o seu papel na arquitetura.
- **Comentários de linha** nos trechos que exigem explicação: o motivo de usar `RETURN_GENERATED_KEYS`, o padrão Post/Redirect/Get, o escape dos coringas do `LIKE`, o `setCharacterEncoding("UTF-8")` e, nas JSPs, o uso de `<c:out>` e de `<c:url>`.

A documentação HTML pode ser gerada com a ferramenta Javadoc do JDK, por exemplo com `mvn javadoc:javadoc` (o resultado fica em `target/reports/apidocs/index.html`).

4.5 Repositório e documentação do projeto

O projeto está publicado no GitHub, em **https://github.com/wiirios/pit-cc**. O arquivo `README.md`, na raiz do repositório, apresenta o sistema e explica como executá-lo. Os documentos de apoio ficam na pasta `docs/`, e o código-fonte em `src/main/`:

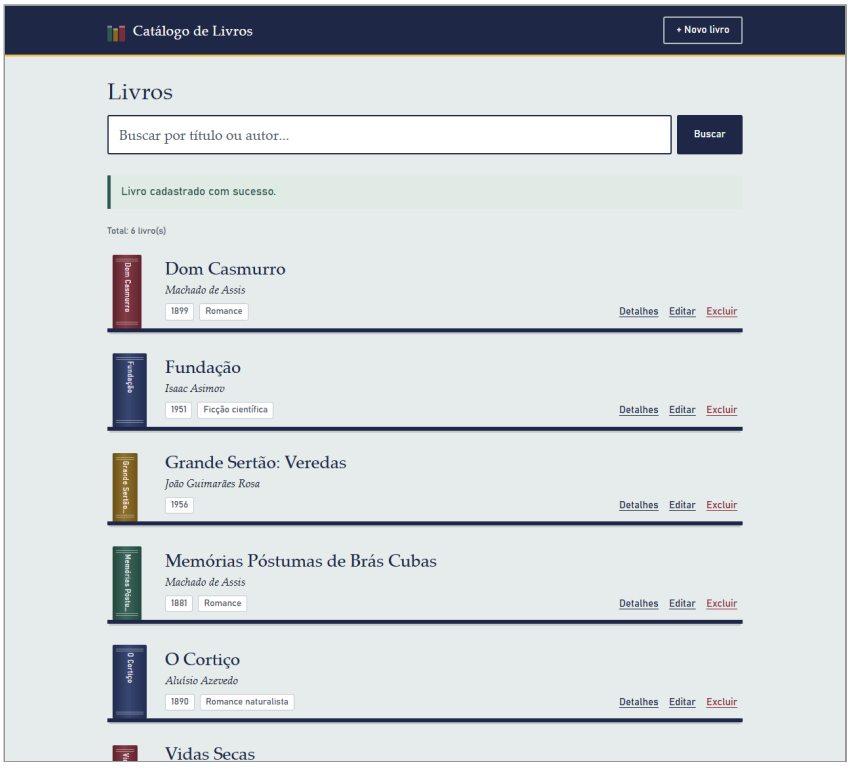
Caminho no projeto	Conteúdo
`README.md`	Apresentação do projeto, tecnologias e instruções de execução
`docs/relatorio-tecnico.pdf`	Este relatório técnico
`docs/relatorio-tecnico.md`	Texto-fonte deste relatório, em Markdown
`docs/modelagem.md`	Diagramas de casos de uso, de classes e DER (Mermaid) e script SQL
`docs/manual-do-usuario.md`	Manual do usuário (reproduzido no Apêndice A)
`docs/roteiro-de-testes.md`	Roteiro de testes manuais do fluxo completo, com resultados esperados
`docs/prints/`	Capturas de tela da aplicação usadas na seção 5
`src/main/java/br/edu/catalogo/`	Código Java: modelo, Servlets, Service, DAO e utilitários
`src/main/webapp/`	JSPs (`WEB-INF/jsp/`), `web.xml` e folha de estilos (`css/estilo.css`)
`src/main/resources/sql/script.sql`	Script de criação da tabela `livro`

5. RESULTADOS

Identidade visual. A interface segue a direção "fichário de biblioteca". Ela usa só HTML e CSS próprios (`css/estilo.css`) e fontes já instaladas no sistema, para rodar offline:

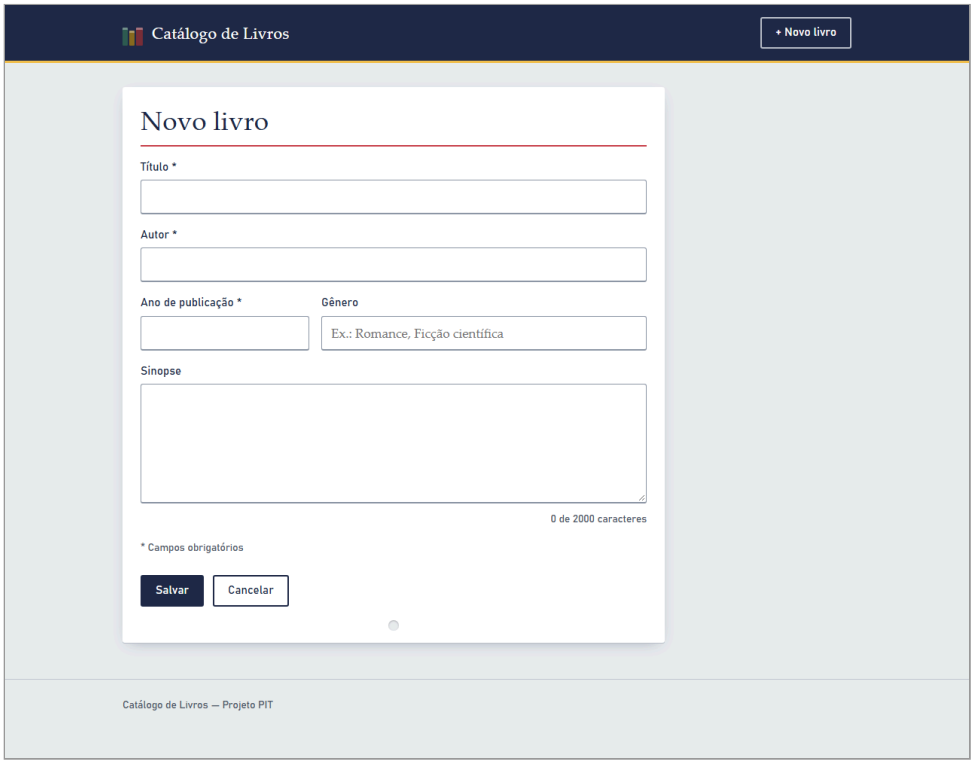
- **Lombadas:** como não há capa, cada livro aparece como uma lombada de tecido desenhada em CSS, com o título na vertical. A cor vem do resto da divisão do id por 6 (classes `lombada-0` a `lombada-5`), então cada livro mantém sempre a mesma cor.
- **Estante:** a listagem é uma lista de livros sobre "prateleiras" (borda grossa sob cada item), com a busca em destaque no topo. A busca sem resultado e o catálogo vazio mostram uma prateleira vazia.
- **Fichas catalográficas:** detalhes, formulário e confirmação de exclusão são cartões com a linha vermelha sob o título, a pauta azul na sinopse e o furo da haste do fichário.
- **Validação:** além do resumo no topo, cada erro aparece logo abaixo do seu campo (borda e fundo vermelhos, `aria-invalid` e `aria-describedby`). A JSP separa as mensagens do `LivroService` pelo início do texto (ex.: "O título…"), sem alterar o Java.
- **Acessibilidade e JavaScript:** labels ligados aos campos, foco visível pelo teclado, contraste adequado e respeito a `prefers-reduced-motion`. O único JavaScript é o contador de caracteres da sinopse; sem ele, o formulário funciona do mesmo jeito.

Figura 4 – Estante (listagem) com a mensagem "Livro cadastrado com sucesso."



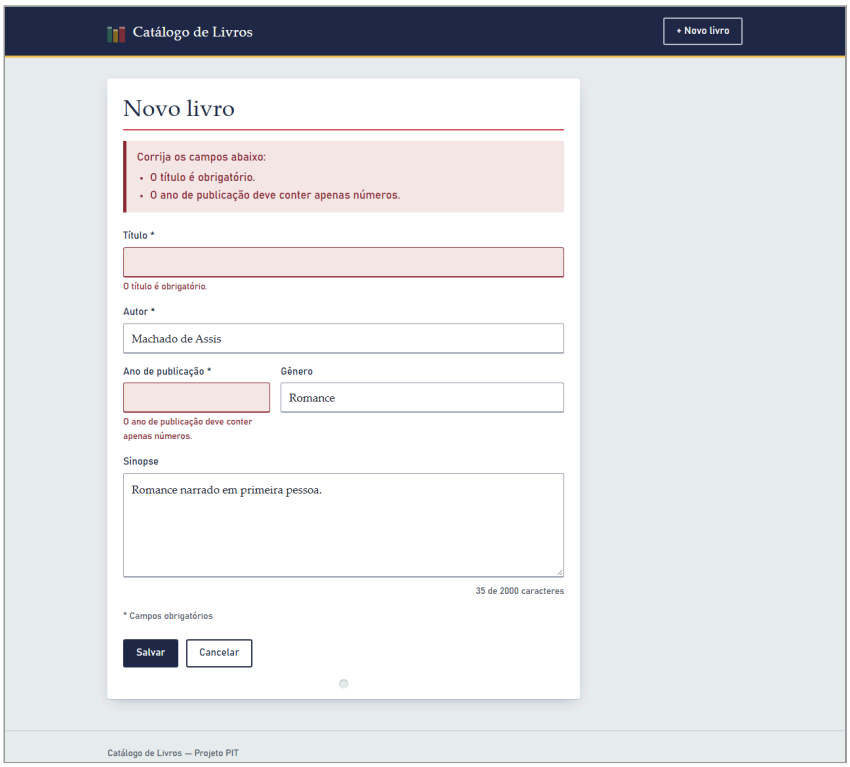
Fonte: elaborado pelo autor (2026).

Figura 5 – Formulário de cadastro vazio (ficha "Novo livro", com o contador da sinopse)



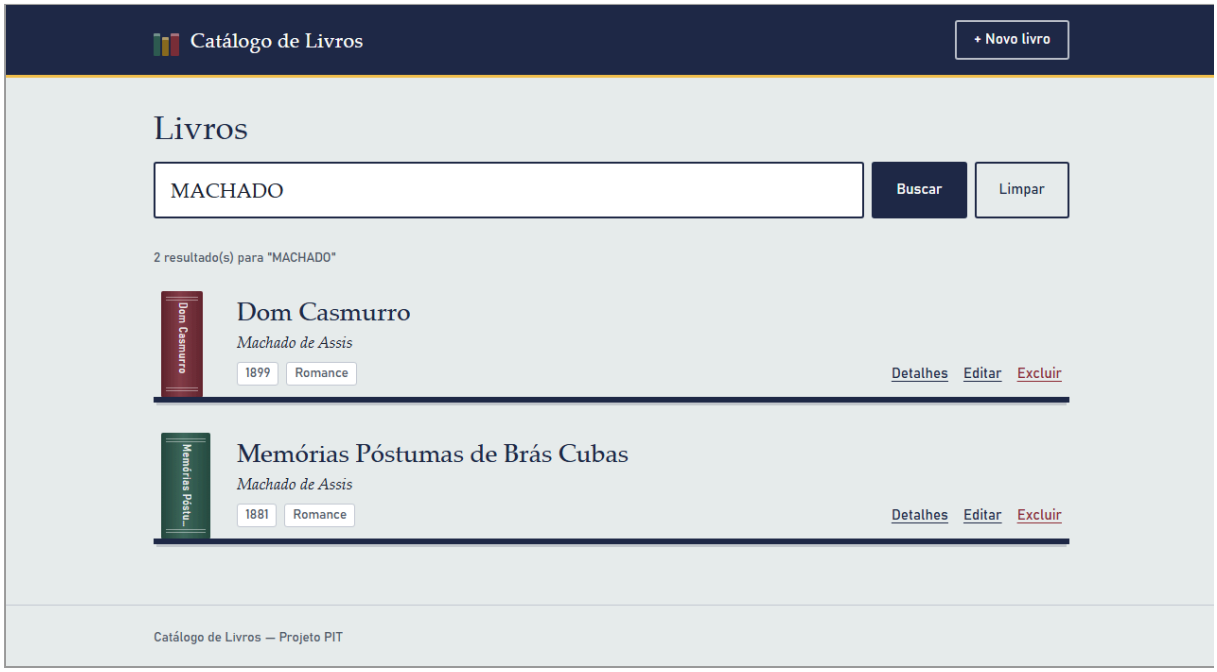
Fonte: elaborado pelo autor (2026).

Figura 6 – Formulário com os erros junto de cada campo (ex.: título vazio e ano com letras)



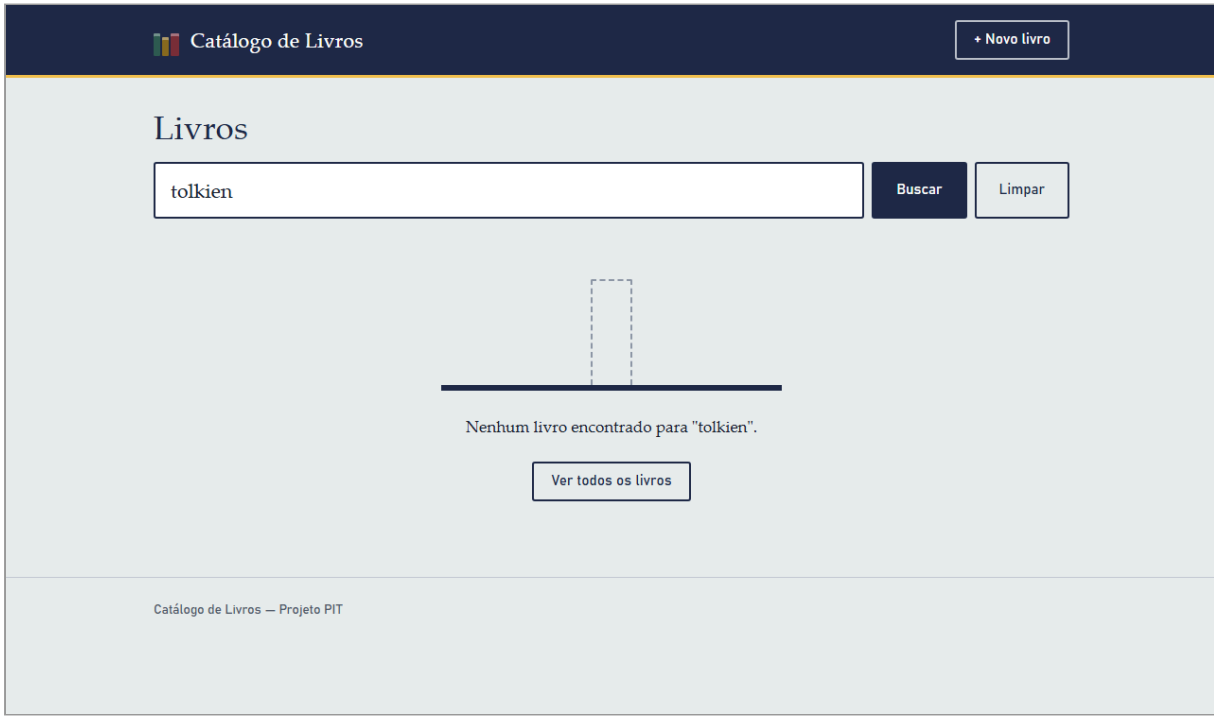
Fonte: elaborado pelo autor (2026).

Figura 7 – Resultado de uma busca por autor (ex.: "MACHADO")



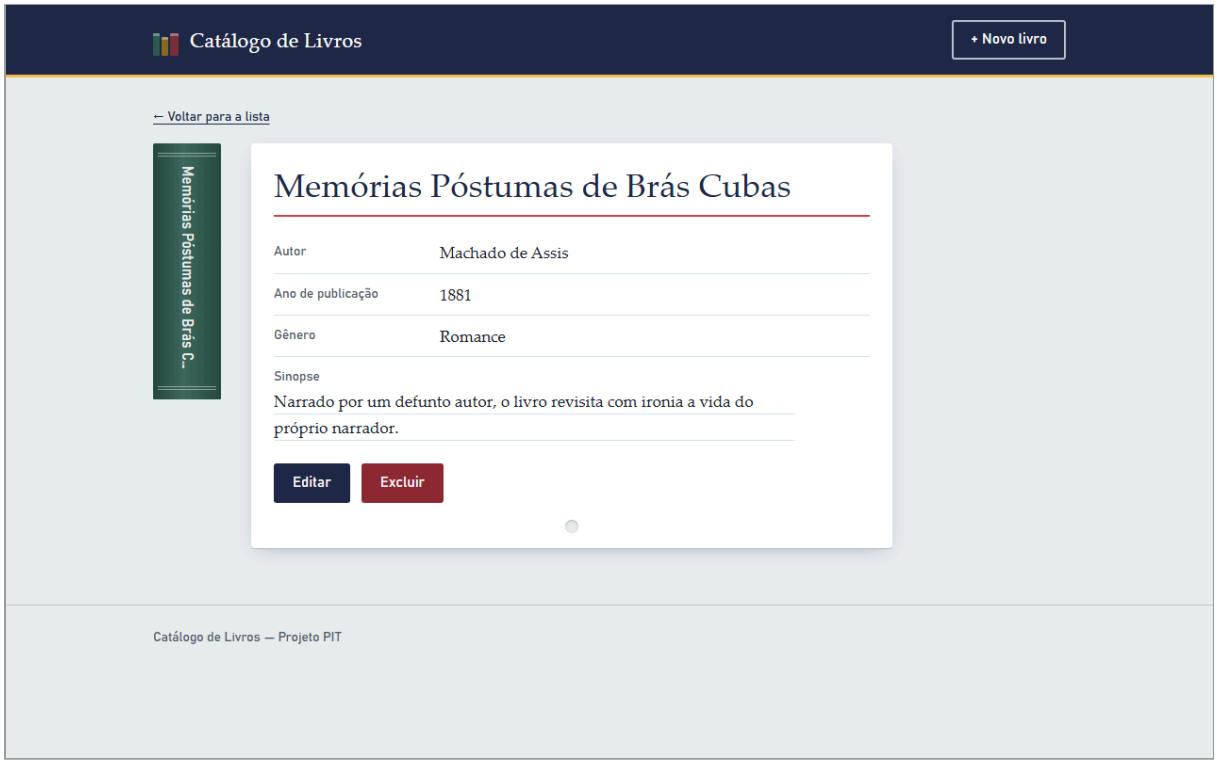
Fonte: elaborado pelo autor (2026).

Figura 8 – Busca sem resultados (prateleira vazia)



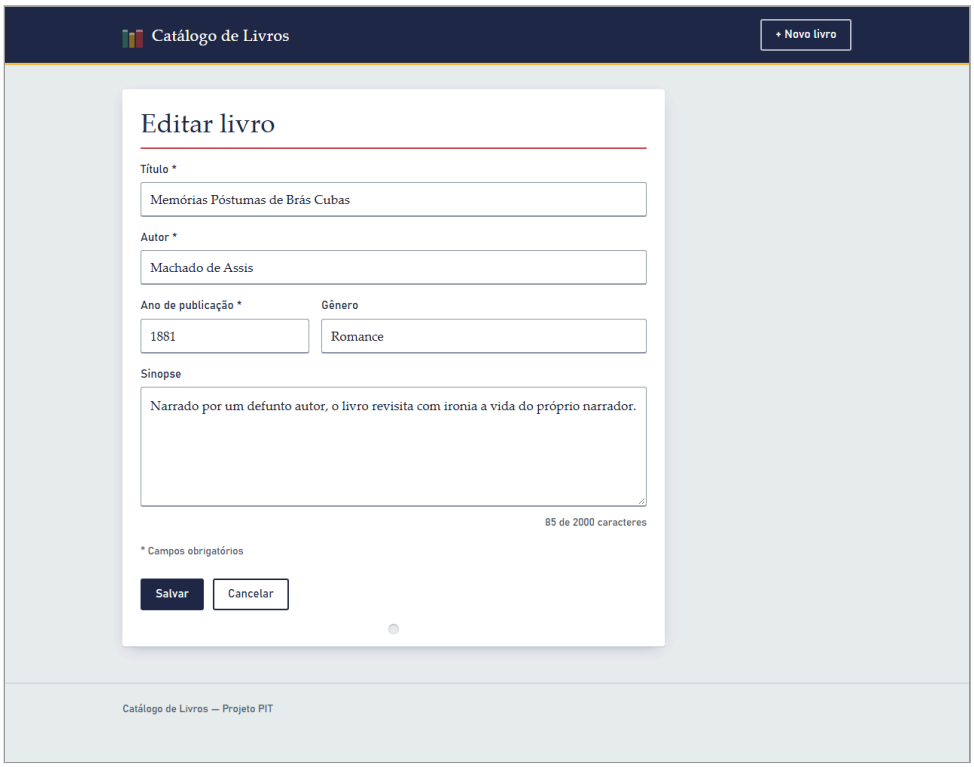
Fonte: elaborado pelo autor (2026).

Figura 9 – Tela de detalhes de um livro (lombada em pé ao lado da ficha)



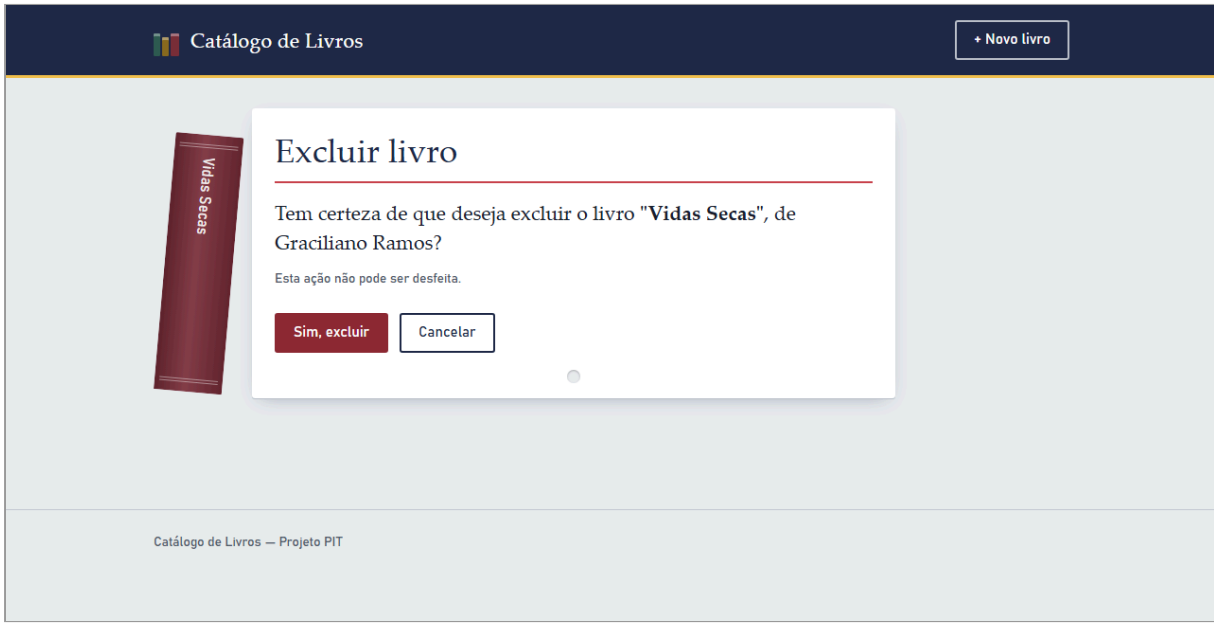
Fonte: elaborado pelo autor (2026).

Figura 10 – Formulário de edição preenchido



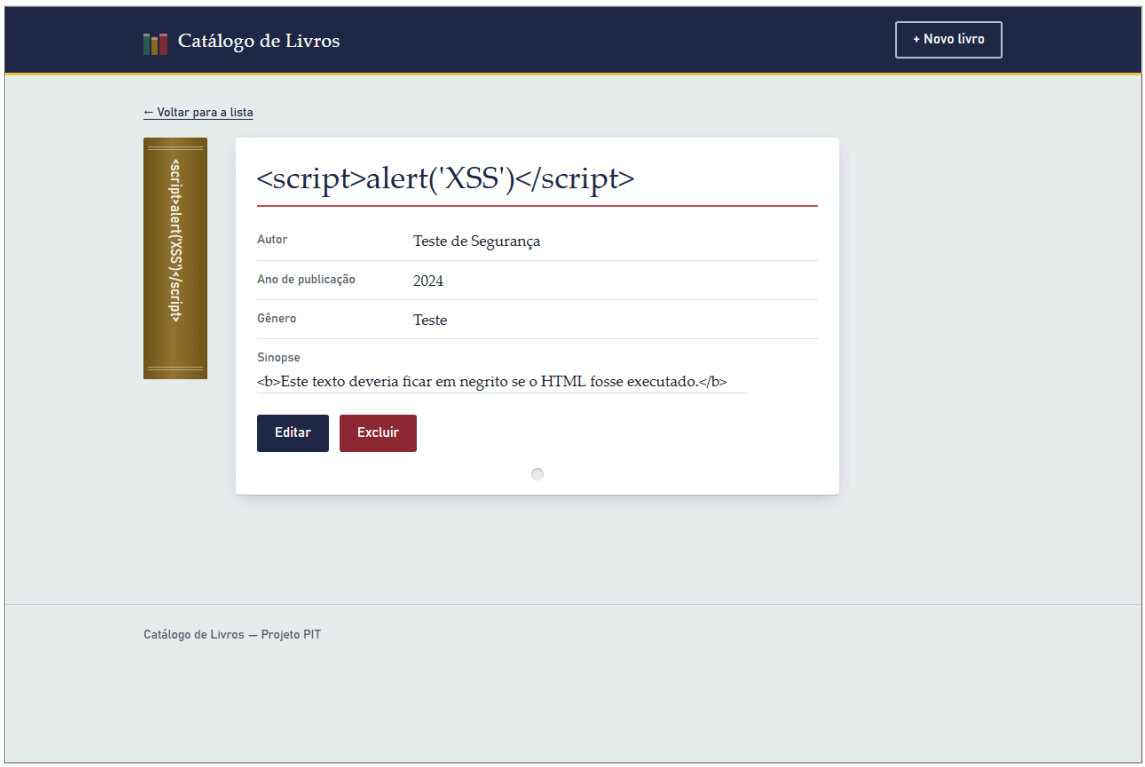
Fonte: elaborado pelo autor (2026).

Figura 11 – Tela de confirmação de exclusão (lombada inclinada, saindo da estante)



Fonte: elaborado pelo autor (2026).

Figura 12 – Livro com <script> no título exibido como texto, inclusive na lombada (teste de XSS)



Fonte: elaborado pelo autor (2026).

Figura 13 – Página "Página não encontrada" (livro caído na prateleira)



Fonte: elaborado pelo autor (2026).

Figura 14 – Estante e tela de detalhes em largura de celular



Fonte: elaborado pelo autor (2026).

6. CONSIDERAÇÕES SOBRE SEGURANÇA

Os pilares considerados foram **integridade** (os dados do catálogo não podem ser alterados indevidamente) e **disponibilidade** (a aplicação não pode "quebrar" com entradas inválidas). **Confidencialidade** tem peso menor, porque o sistema não guarda dados pessoais nem tem login (ISO/IEC 27001, conforme o material da disciplina).

Ameaça / risco	**Medida implementada**	**Onde**
SQL Injection	`PreparedStatement` com parâmetros `?` em **todas** as consultas, inclusive na busca; nenhum dado do usuário é concatenado em SQL	`LivroDAO` (todos os métodos)
Coringas do LIKE	`%` e `_` digitados são escapados (`ESCAPE '\'`)	`LivroDAO.escaparLike()`
XSS	Toda saída de dados nas JSPs passa por `<c:out>`, que converte `<`, `>`, `"` e `'` em entidades HTML	Todas as JSPs
Dados inválidos	Validação **no servidor** (obrigatórios, ano numérico e na faixa, tamanhos máximos); o `required`/`maxlength` do HTML é só ajuda ao usuário, pois pode ser burlado	`LivroService.validar()`
Estouro de tamanho no banco	Limites validados = tamanhos das colunas `VARCHAR`; `NOT NULL` no banco	`LivroService`, `script.sql`
Exposição de detalhes técnicos	Exceções tratadas com try-catch-finally; mensagens amigáveis; páginas de erro próprias para 400, 404, 500, qualquer exceção e qualquer outro código (ex.: 405)	`LivroDAO`, Servlets, `web.xml`, `erro.jsp`
Exclusão acidental ou por link	Excluir só por POST, depois de uma página de confirmação	`ExcluirLivroServlet`
Manipulação do id na URL	Id convertido com segurança (texto, zero ou negativo viram "não encontrado")	`Parametros.lerId()`
Mensagem injetada pela URL	O parâmetro `msg` só aceita códigos fixos; o texto exibido vem do código Java	`ListarLivrosServlet.traduzirMensagem()`
Id de sessão vazando na URL	Sessão rastreada só por cookie (`tracking-mode COOKIE`), sem `;jsessionid=` nos links	`web.xml`
Acesso direto às telas	JSPs dentro de `WEB-INF`, inacessíveis sem passar por um Servlet	estrutura do projeto

Limitações conhecidas (sugestões para trabalhos futuros): não há autenticação, então qualquer pessoa com acesso ao endereço pode alterar o catálogo, e não há proteção contra CSRF nos formulários. Para um catálogo pessoal rodando em `localhost`, o risco é baixo.

7. CONCLUSÕES

O desenvolvimento do Catálogo Simples de Livros cumpriu o objetivo proposto pela disciplina: entregar uma aplicação web funcional, construída com Java, Servlets, JSP e JDBC, capaz de cadastrar, listar, detalhar, editar, excluir e buscar livros, com os dados persistidos em um banco relacional. Mais do que um conjunto de telas, o projeto permitiu vivenciar, em escala reduzida, o ciclo completo de construção de um software: do planejamento e da modelagem à implementação, aos testes e à documentação.

Dificuldades encontradas. Os principais obstáculos surgiram em detalhes que, embora pequenos, comprometiam a experiência do usuário ou a segurança da aplicação. A acentuação, por exemplo, só passou a ser tratada corretamente depois que a codificação UTF-8 foi configurada tanto na leitura das requisições quanto nas páginas JSP. Da mesma forma, a redução do escopo para um catálogo exclusivamente de livros exigiu recriar o banco de desenvolvimento, uma vez que o comando `CREATE TABLE IF NOT EXISTS` não altera tabelas já existentes. Houve ainda ajustes de segurança menos evidentes, como impedir que o identificador de sessão aparecesse nos links e substituir as páginas de erro padrão do servidor, que expunham informações técnicas, por mensagens claras ao usuário.

Aprendizados. O principal aprendizado foi compreender que segurança e organização não são etapas finais, mas decisões que acompanham o projeto desde o início. A validação no servidor mostrou-se indispensável, já que a validação do navegador pode ser facilmente contornada. O uso de `PreparedStatement` evidenciou, na prática, como a separação entre comando e dados neutraliza ataques de SQL Injection, assim como a saída escapada com `<c:out>` impede a execução de scripts maliciosos (XSS). Além disso, a divisão em camadas (Servlet, Service e DAO) revelou seu valor concreto: quando o escopo mudou, as alterações ficaram restritas aos pontos certos, sem efeitos colaterais no restante do sistema. Por fim, o tratamento de exceções com try-catch-finally e o padrão Post/Redirect/Get reforçaram a importância de prever o comportamento da aplicação também diante de erros e de ações repetidas do usuário.

Trabalhos futuros. Como evolução natural, o catálogo pode incorporar as extensões sugeridas no enunciado (avaliação por estrelas, imagem de capa e filtro por gênero), além de testes automatizados com JUnit para as camadas de serviço e de acesso a dados. Caso venha a ser compartilhado entre vários usuários, será necessário implementar autenticação e proteção contra CSRF, o que consolidaria o sistema como uma ferramenta segura também fora do ambiente local.

8. REFERÊNCIAS

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2007.

FOWLER, M. **UML distilled**: a brief guide to the standard object modeling language. 3rd ed. Boston: Addison-Wesley Professional, 2004.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 27001:2013**: Information technology — Security techniques — Information security management systems — Requirements. 1st ed. Geneva: ISO, 2013.

JAVA servlets tutorial: Java JSP tutorial: Java server-side programming for beginners. [*S. l.*: *s. n.*], 2020. 1 vídeo (27 min). Publicado pelo canal Simplilearn. Disponível em: https://www.youtube.com/watch?v=7AIjcZMo-V4. Acesso em: 22 out. 2026.

LARMER, J.; MERGENDOLLER, J.; BOSS, S. **Setting the standard for project-based learning**: a proven approach to rigorous classroom instruction. 1st ed. Alexandria: ASCD, 2015.

MCGRAW, G. **Software security**: building security in. 1st ed. Upper Saddle River: Addison-Wesley, 2006.

MYERS, G. J.; SANDLER, C.; BADGETT, T. **The art of software testing**. 3rd ed. Hoboken: John Wiley & Sons, 2012.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

APÊNDICE A — MANUAL DO USUÁRIO

1. O que é

Aplicação web para cadastrar e consultar livros. Para cada livro, o catálogo guarda título, autor, ano de publicação, gênero e sinopse. Os dados ficam salvos em um banco de dados e continuam lá depois que o servidor é desligado.

2. Como instalar e executar

Requisitos

- **JDK 17 ou mais recente** (comando `java -version`)
- **Apache Maven 3.9 ou mais recente** (comando `mvn -v`)
- Navegador web (Chrome, Edge, Firefox…)

Não é preciso instalar banco de dados nem servidor: o banco H2 e o servidor Jetty são baixados pelo Maven na primeira execução, por isso ela exige internet e demora um pouco mais.

Passo a passo

1. Abra um terminal na pasta do projeto (a pasta que contém o arquivo `pom.xml`).
2. Execute:

```
mvn jetty:run
```

3. Espere aparecer no terminal a linha `Started oejs.Server`.
4. Abra no navegador: **http://localhost:8080**
5. Para desligar o servidor, volte ao terminal e pressione **Ctrl + C**.

Onde ficam os dados

O banco é criado sozinho no arquivo `dados/catalogo.mv.db`, dentro da pasta do projeto.

- Para **apagar todos os livros** e começar do zero: desligue o servidor e apague a pasta `dados`. Na próxima execução, a tabela é recriada vazia.
- Se aparecer erro de banco "em uso", verifique se não há outro `mvn jetty:run` aberto: o H2 em modo arquivo só aceita um servidor por vez.

- Se a porta 8080 estiver ocupada por outro programa, feche esse programa e rode de novo.

3. Como usar

Tela inicial: lista de livros

Ao abrir http://localhost:8080 você é levado à lista de livros, em ordem alfabética de título. Em cada linha há três links: **Detalhes**, **Editar** e **Excluir**. No celular, cada livro aparece como um cartão.

Cadastrar um livro

1. Clique em **+ Novo livro** (canto superior direito).
2. Preencha os campos. Os marcados com ***** são obrigatórios:
    - **Título*** — até 255 caracteres
    - **Autor*** — até 255 caracteres
    - **Ano de publicação*** — apenas números (de 1 até o ano atual + 5)
    - **Gênero** — opcional, até 100 caracteres (ex.: Romance, Ficção científica)
    - **Sinopse** — opcional, até 2000 caracteres (as quebras de linha são mantidas)
3. Clique em **Salvar**. Você volta à lista com a mensagem "Livro cadastrado com sucesso."

Se algum campo estiver errado, o formulário volta com a lista do que corrigir e com o que você já tinha digitado.

Buscar

Digite parte do **título ou do nome do autor** no campo de busca e clique em **Buscar**.

- A busca não diferencia maiúsculas de minúsculas ("machado" acha "Machado de Assis").
- Os acentos precisam ser digitados ("sertão" acha "Grande Sertão: Veredas"; "sertao" não acha).
- Clique em **Limpar** para voltar à lista completa.

Ver detalhes

Clique no título do livro ou em **Detalhes** para ver todos os campos, inclusive a sinopse.

Editar

Na lista ou na tela de detalhes, clique em **Editar**. O formulário abre preenchido; altere o que quiser e clique em **Salvar**. Você vai para a tela de detalhes com a mensagem "Alterações salvas com sucesso." Para desistir, clique em **Cancelar**.

Excluir

Clique em **Excluir**. O sistema pergunta se você tem certeza e mostra o título e o autor do livro.

- **Sim, excluir** apaga o livro definitivamente (não é possível desfazer).
- **Cancelar** volta para os detalhes sem apagar nada.

4. Mensagens que podem aparecer

Mensagem	O que significa
"Livro não encontrado. Ele pode ter sido excluído."	O endereço aponta para um livro que não existe (ou que foi excluído)
"Página não encontrada"	O endereço digitado não existe no sistema
"Algo deu errado" / "Não foi possível concluir a operação"	Erro inesperado; tente de novo e, se continuar, reinicie o servidor
"Nenhum livro encontrado para …"	A busca não encontrou nenhum título ou autor com esse texto